ARCHITECTURE DESIGN

Hospital Appointment and Patient Record System

1. Introduction

The Hospital Appointment and Patient Record System is a web-based application designed to manage hospital operations such as appointment scheduling, patient record maintenance, doctor queue management, and lab services. The system provides a centralized platform where patients can book appointments online and administrators can efficiently manage hospital resources.

The application is built using the MERN stack and follows the Model View Controller (MVC) architecture in the backend to ensure modularity, scalability, and maintainability.

2. System Objectives

The system is designed to improve healthcare accessibility and hospital management efficiency.

- Provide an easy and efficient online appointment booking system
- Maintain digital patient health records
- Reduce waiting time using a queue display system
- Integrate lab services with digital report access
- Enable role-based dashboards for patients, doctors, and administrators
- Ensure scalability and organized system design

3. Architectural Style (MVC Pattern)

The backend follows the MVC design pattern to separate concerns and improve system organization.

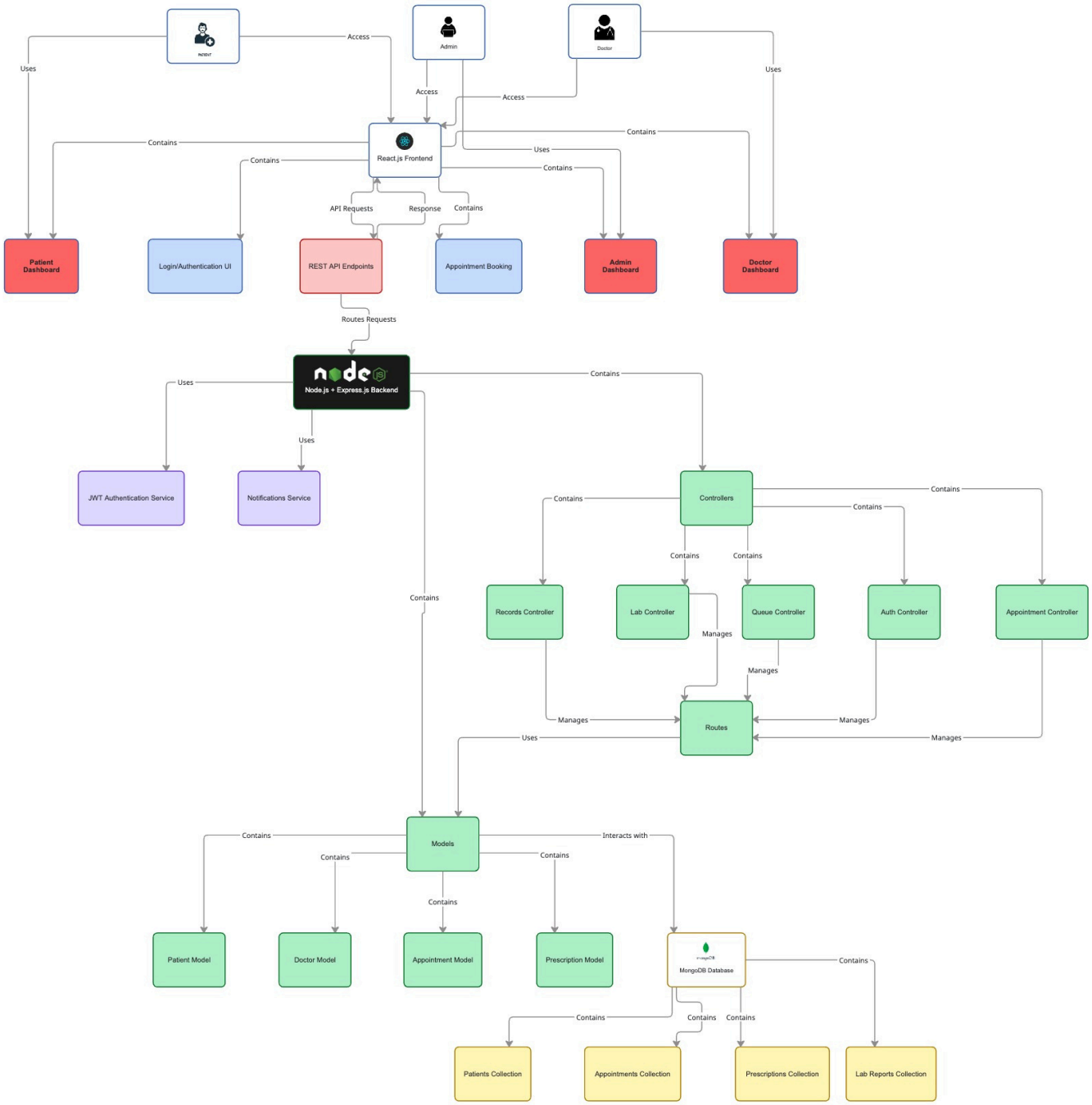
- **Model** – Handles database structure and operations
- **View** – Represents the user interface (React frontend)
- **Controller** – Handles business logic and request processing

4. High-Level Architecture

The system consists of:

- **Frontend (React.js)** – User interface for patients, doctors, and admins
- **Backend (Node.js + Express.js)** – Handles logic and API requests
- **Database (MongoDB)** – Stores users, appointments, health records, lab reports

Users interact with the frontend, which communicates with the backend through REST APIs. The backend processes requests and interacts with the database.



5. MVC Component Design

5.1 Model (Data Layer)

The model defines data structures and manages database operations.

- Represents entities such as **User, Appointment, HealthRecord, LabReport, Doctor, Feedback**
- Uses Mongoose schemas
- Performs CRUD operations
- Maintains relationships between patients, doctors, and appointments

5.2 View (Presentation Layer)

The view is responsible for displaying the user interface and handling interactions.

- Displays appointment booking, health records, and lab reports
- Provides forms for login, registration, and booking
- Sends API requests to backend
- Updates UI dynamically based on user role

Main Pages:

- Patient Dashboard
- Doctor Dashboard
- Admin Dashboard
- Appointment Booking Page
- Health Records Page
- Lab Services Page
- Ask Expert / Feedback Page

5.3 Controller (Application Logic)

The controller processes user requests and applies business logic.

- Handles appointment booking and cancellation
- Manages patient queue and token assignment
- Processes health record creation and updates
- Manages lab test booking and report upload
- Validates user role and access permissions
- Handles doctor availability and staff schedules (admin)

6. Request Response Flow

1. User logs in and selects an action (e.g., book appointment)
2. Frontend sends request to backend API
3. JWT middleware validates the token and user role
4. Request is handled by Express routes
5. Routes forward request to the appropriate controller
6. Controller processes logic (booking / queue / records)
7. Controller interacts with the model
8. Model performs database operations on MongoDB
9. Database returns result to the model
10. Controller sends JSON response
11. Frontend updates UI accordingly

7. API Design

The system uses RESTful APIs:

- POST /api/auth/register — Register user
- POST /api/auth/login — Login user
- GET /api/doctors — Get all doctors
- POST /api/appointments — Book appointment
- GET /api/appointments/queue — Get queue token and wait time
- GET /api/records/:patientId — Get health records
- POST /api/records — Create health record
- POST /api/labs/book — Book lab test
- PUT /api/labs/:id/upload — Upload lab report
- POST /api/feedback — Submit doctor feedback
- PUT /api/admin/beds — Update bed occupancy
- PUT /api/admin/queue — Update queue order

8. Security Architecture

The system ensures secure access and data protection:

- Authentication using JWT (JSON Web Tokens)
- Password hashing using bcrypt
- Role-based access control (Patient / Doctor / Admin)
- Protected routes using middleware
- Health record access restricted — admin cannot view patient records

9. Scalability and Maintainability

The system is designed for future improvements:

- Modular MVC structure
- Separate frontend and backend
- Reusable React components
- Easy addition of new features

Future enhancements may include:

- Email and SMS notifications for appointment reminders
- Real-time queue updates using WebSockets
- Telemedicine / video consultation integration
- Mobile application using React Native

10. Conclusion

The Hospital Appointment and Patient Record System is designed using a scalable and modular architecture based on the MERN stack and MVC pattern. The system ensures efficient management of appointments, patient records, lab services, and hospital operations.

By digitizing hospital operations and integrating appointment booking, health records, and queue management into one platform, the system improves accuracy, reduces manual work, and enhances patient experience. The architecture provides a strong foundation for future enhancements and real-world implementation.

Prepared By:

K.Roshini - B23CS107

K.Pranathi - B23EC146

K.Poojitha - B23IT082